TECHNICAL REPORT

The Transformer *Encoder*

A comprehensive technical deep dive into the architecture, mathematics, and mechanics of the encoder component in the Transformer model — from raw token embeddings to contextualised representations.

TOPIC	ORIGINAL PAPER	DATE
Deep Learning / NLP Architectures	Attention Is All You Need (2017)	June 2026

CONTENTS

01 OVERVIEW & HISTORICAL CONTEXT

Why the Encoder Exists

The Transformer encoder converts a sequence of discrete tokens into a sequence of continuous, context-aware vector representations — the "reading" half of the original Transformer.

Before 2017, sequence modelling was dominated by recurrent architectures — LSTMs and GRUs — which processed tokens one by one, left to right. This imposed an inherent bottleneck: information from the beginning of a long sequence had to flow through hundreds of time-steps before influencing the final representation. Long-range dependencies were difficult to capture, and the sequential processing prevented effective GPU parallelism.

Vaswani et al. (2017) broke this paradigm with the Transformer. Rather than processing tokens in order, every token in the encoder attends to every other token simultaneously, in a single operation. The encoder builds a rich, dense representation of the entire input in parallel — a representation that captures both local syntax and long-range semantic relationships.

Bidirectional Context

Every token can attend to every other token, both left and right, simultaneously. No sequential bottleneck.

Parallel Processing

All attention computations happen in matrix form, enabling massive GPU parallelism absent in RNNs.

Composable Layers

Multiple identical encoder layers are stacked, each refining the representation learned by the previous one.

Fixed-Dimension Output

Input and output share the same dimensionality d_{model}, making layers easy to stack.

The Encoder's Role in the Larger Architecture

In the original seq2seq Transformer (e.g. for machine translation), the encoder processes the source sequence and produces a set of representations. The decoder then attends to those representations while generating the target sequence. However, encoder-only models — the most famous being BERT — have proven enormously effective for understanding tasks: classification, named entity recognition, question answering, and semantic similarity.

The encoder produces one vector per input token, each of dimension `d_model` (512 in the original paper, 768 in BERT-base, 1024 in BERT-large). Together, these vectors form a matrix of shape `(sequence_length, d_model)` that is the encoder's output.

---

02 INPUT PIPELINE

Embeddings & Positional Encoding

Before a single attention computation takes place, raw token IDs must be transformed into dense, position-aware floating-point vectors that the model can work with.

Token Embeddings

Each token in the vocabulary is mapped to a trainable vector in $\mathbb{R}^{d\_model}$ via an embedding matrix $E \in \mathbb{R}^{|V| \times d\_model}$. This is a standard learned lookup table. In the original paper, these weights are shared with the output projection in the decoder (weight tying), which acts as a regulariser and reduces parameter count.

$$x_i = \mathbf{E}[\text{token}_i] \cdot \sqrt{d_{\text{model}}}$$

Token embedding scaled by $\sqrt{d_{\text{model}}}$ to prevent embedding magnitudes from overwhelming positional encodings.

The scaling factor $\sqrt{d\_model}$ is important: positional encodings are fixed values bounded in [-1, 1], while embedding weights can grow arbitrarily during training. Without scaling, positional information could become negligible relative to the semantic content — or overwhelm it entirely.

Positional Encoding

Unlike RNNs, the self-attention mechanism has no inherent notion of order — it treats its input as a set, not a sequence. Positional encodings inject order information by adding a unique signal to each token's embedding based on its position.

The original paper uses fixed, sinusoidal encodings:

$$\mathrm{PE}(\mathrm{pos},\, 2i) = \sin\left(\frac{\mathrm{pos}}{10000^{\,2i/d_{\mathrm{model}}}}\right)$$

$$\mathrm{PE}(\mathrm{pos},\, 2i+1) = \cos\left(\frac{\mathrm{pos}}{10000^{\,2i/d_{\mathrm{model}}}}\right)$$

pos = *position in the sequence;* i = *dimension index. Even dimensions use sine, odd use cosine.*

Why sinusoids?

The choice of sinusoidal functions was deliberate. For any fixed offset *k*, `PE(pos+k)` can be expressed as a linear transformation of `PE(pos)`. This means the model could theoretically learn to attend by relative position, not just absolute position.

Furthermore, sinusoids with different frequencies allow the model to distinguish positions across long sequences: low-frequency dimensions vary slowly (useful for long-range patterns), high-frequency dimensions vary rapidly (useful for local, adjacent-token patterns).

Learned vs. Fixed

BERT and most modern encoder-only models replace sinusoidal PEs with *learned* positional embeddings — a trainable matrix of shape `(max_seq_len, d_model)`. Experiments show similar final performance, but learned embeddings cannot generalise to sequences longer than those seen during training.

More recent variants use *relative positional encodings* (e.g. RoPE, ALiBi, T5's relative biases), which encode the distance between two tokens rather than their absolute positions — improving extrapolation to unseen sequence lengths.

```
INPUT CONSTRUCTION
```

The final encoder input for token at position *pos* is simply the element-wise sum: $\mathbf{X}[\text{pos}] = \text{TokenEmbedding}(\text{token\_id}) \times \sqrt{d_{\text{model}}} + \text{PE}(\text{pos})$. This produces a matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$ that feeds into the first encoder layer.

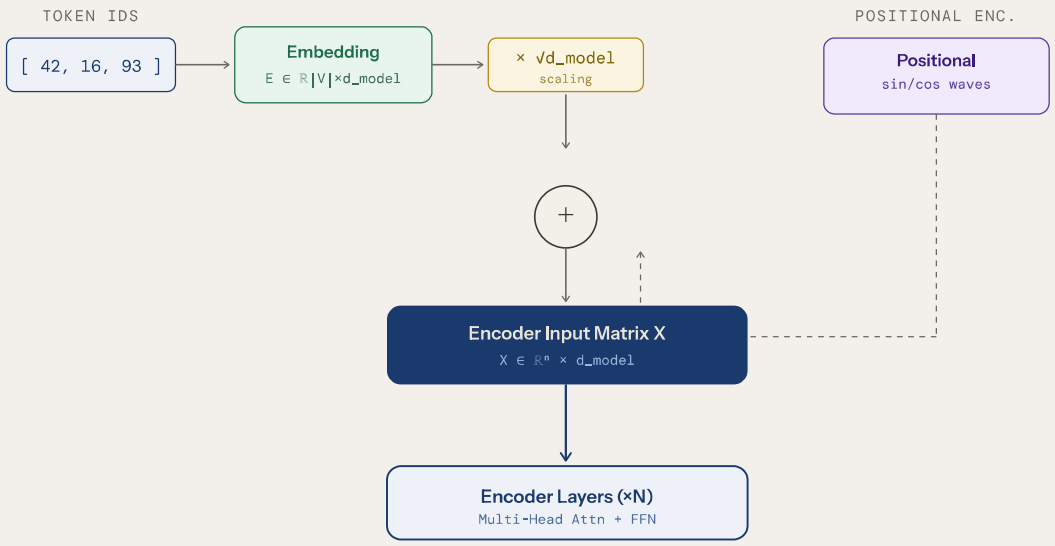

Figure 1 — The encoder input pipeline. Token IDs are looked up in the embedding matrix, scaled, and summed with sinusoidal positional encodings before entering the stack of encoder layers.

03 MULTI-HEAD SELF-ATTENTION

The Core Mechanism

Self-attention is the engine that allows every token to dynamically "look at" every other token and aggregate their information — weighted by learned relevance.

Scaled Dot-Product Attention

The fundamental building block is *Scaled Dot-Product Attention*. Given an input matrix `X ∈ ℝ^{n × d_model}`, the mechanism first projects it into three separate spaces — Queries, Keys, and Values — via learned weight matrices:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

$$\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k} \text{ and } \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

The attention output is then computed as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

$$d_k = 64 \text{ in the original paper } (= d_{\text{model}}/h = 512/8)$$

1

Compute Raw Attention Scores

Multiply Q by K^T to get an `n × n` score matrix. The entry at position `(i, j)` is the dot product between query vector *i* and key vector *j* — a measure of "how relevant is token j to token i?". High dot products mean high relevance.

2

Scale by √d_k

In high-dimensional spaces, dot products tend to grow large in magnitude, pushing softmax into regions with very small gradients (the "vanishing gradient of softmax" problem). Dividing by `√d_k` keeps the values in a well-behaved range before softmax is applied. This is the "Scaled" in Scaled Dot-Product Attention.

3

Apply Softmax (Row-Wise)

Softmax is applied row-by-row, converting each row into a probability distribution over the *n* tokens. For token *i*, this gives a set of attention weights — non-negative and summing to 1 — describing how much attention token *i* should pay to each other token (including itself).

4

Weighted Sum of Values

Multiply the attention weight matrix by V. Each row of the output is a weighted sum of all value vectors, where the weights reflect relevance. The result is a new matrix of shape `n × d_v`, where each row is an attention-pooled contextual representation of the corresponding token.

Multi-Head Attention

A single attention head can only capture one type of relationship at a time. Multi-Head Attention runs *h* independent attention heads in parallel, each with its own learned projections, then concatenates and re-projects their outputs:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, \mathbf{W}_O$$

$$\text{where head}_i = \text{Attention}\left(\mathbf{Q}\mathbf{W}_Q^{(i)},\, \mathbf{K}\mathbf{W}_K^{(i)},\, \mathbf{V}\mathbf{W}_V^{(i)}\right)$$

$\mathbf{W}_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. *In the base model:* $h = 8,\ d_k = d_v = 64,\ hd_v = 512 = d_{\text{model}}$.

WHY MULTIPLE HEADS?

Each attention head learns to focus on a different aspect of token relationships. Empirical analyses show that different heads capture different linguistic phenomena: one head may focus on syntactic dependencies (subject-verb agreement), another on coreference, another on positional proximity. By concatenating all heads, the model obtains a rich, multi-faceted contextualised representation.

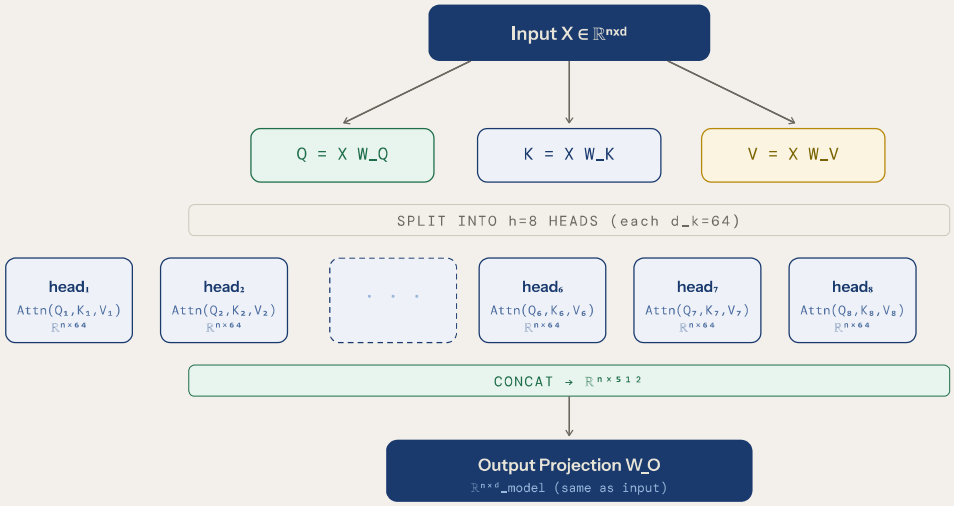

Figure 2 — Multi-Head Attention data flow. Eight parallel heads each see their own projected Q, K, V slices; their outputs are concatenated and projected back to d_model.

Attention Complexity

The attention score matrix $\mathbf{QK}^\top$ has shape $n \times n$. This means the time and memory complexity of self-attention scale as $\mathcal{O}(n^2)$ with sequence length. For long sequences (e.g. documents), this quadratic scaling is a significant bottleneck — motivating sparse attention variants (Longformer, BigBird, Flash-Attention), linear approximations (Performer, Linformer), and state-space models.

PARAMETER	BASE MODEL VALUE	BERT-BASE	BERT-LARGE	NOTES
d_model	512	768	1024	Embedding / hidden dimension
Heads (h)	8	12	16	Number of attention heads
d_k = d_v	64	64	64	d_model / h; same for Q, K, V

PARAMETER	BASE MODEL VALUE	BERT-BASE	BERT-LARGE	NOTES
QKV params	3 × 512 × 512 = 786K	~2.36M	~4.19M	Per encoder layer
Output proj	512 × 512 = 262K	589K	1.05M	W_O per layer

04 POSITION-WISE FEED-FORWARD NETWORK

The FFN Sub-Layer

After each attention sub-layer, every token's representation passes through an identical, independently-applied two-layer MLP — the position-wise feed-forward network.

The term "position-wise" means the same MLP is applied to each of the *n* token positions independently — there is no interaction between tokens at this stage. This is in contrast to the attention sub-layer, which explicitly mixes information across positions. The FFN can be thought of as applying a pointwise nonlinear transformation to each token's representation.

$$\mathrm{FFN}(\mathbf{x}) = \max(0,\ \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

$\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$. *Original:* $d_{\text{ff}} = 2048$ *(4× expansion). BERT-base:* $d_{\text{ff}} = 3072$.

Architecture

Role and Interpretation

The FFN expands the representation by a factor of 4 (to `d_ff = 4 × d_model`) using W_1, applies a ReLU non-linearity, then projects back down to `d_model` using W_2. This bottleneck-then-expansion structure is inspired by the MLPs used in earlier feedforward networks.

Modern variants often replace ReLU with GELU (Gaussian Error Linear Unit), which approximates a smooth gating function and is used in BERT, GPT, and most current Transformers.

The FFN sub-layer is a large fraction of the model's parameters. In BERT-base, the two FFN weight matrices per layer account for roughly $2 \times 768 \times 3072 \approx 4.7$M parameters per layer.

Research has shown that the FFN layers behave like key-value memory stores — each row of W_1 acts as a "key pattern" and the corresponding row of W_2 as a "value". Factual knowledge appears to be stored largely in the FFN weights, while attention handles information routing.

> GELU VS RELU
>
> $\text{GELU}(x) = x \cdot \Phi(x)$ where Φ is the CDF of the standard normal distribution. Unlike ReLU's hard zero cutoff, GELU attenuates small negative inputs smoothly, reducing the "dead neuron" problem. In practice GELU outperforms ReLU on most NLP benchmarks and is the default activation in BERT, GPT-2, RoBERTa, and others.

ACTIVATION	FORMULA	USED IN	PROPERTIES
ReLU	$\max(0,\, x)$	Original Transformer	Simple, sparse activations; dead neurons possible
GELU	$x \cdot \Phi(x)$	BERT, GPT-2, RoBERTa	Smooth, performs best on NLP benchmarks
SwiGLU	$x \cdot \sigma(\beta x) \otimes x\mathbf{W}$	PaLM, LLaMA, Gemma	Gated variant; SOTA on LLM benchmarks
GLU	$(x\mathbf{W}) \otimes \sigma(x\mathbf{V})$	Various T5 variants	Gating mechanism; expressivity gain

Stabilising Training

Two simple but critical components prevent gradient pathologies and enable training of deep networks: residual (skip) connections and Layer Normalisation.

Residual Connections

Each sub-layer (attention and FFN) in the encoder is wrapped with a residual connection. Rather than simply passing the output of the sub-layer to the next layer, the input to the sub-layer is added back to its output:

$$\text{Output} = \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$$

$\mathbf{x}$ is the input to the sub-layer; $\text{Sublayer}(\mathbf{x})$ is Multi-Head Attention or FFN. This is the Post-LN formulation. Pre-LN reverses the order.

Residual connections, introduced in ResNets (He et al., 2015), are crucial for training deep networks. They provide a direct gradient pathway from later layers all the way back to earlier layers, preventing vanishing gradients. Empirically, without residual connections, Transformers with more than a few layers fail to train effectively.

Layer Normalisation

Layer Normalisation (Ba et al., 2016) normalises the activations across the feature dimension (i.e., across the `d_model` values for a single token, not across the batch). This is in contrast to Batch Normalisation, which normalises across the batch dimension:

$$\text{LayerNorm}(\mathbf{x}) = \gamma \cdot \frac{\mathbf{x} - \mu}{\sigma + \varepsilon} + \beta$$

μ and σ are the mean and std of $\mathbf{x}$ across d_{model}; γ and β are learnable scale and bias; $\varepsilon \approx 10^{-6}$ for numerical stability.

Post-LN (Original)

The original "Attention Is All You Need" paper applied LayerNorm after the residual addition: `LayerNorm(x + Sublayer(x))`. This is "Post-LN" and is what the formulation above describes. Post-LN tends to produce better final accuracy but can be unstable at the start of training, requiring careful warm-up schedules.

Pre-LN (Common in Modern Models)

Modern models often use "Pre-LN": `x + Sublayer(LayerNorm(x))` — normalising the input before the sub-layer rather than after. Pre-LN is more stable at initialisation and can train without warm-up, though it sometimes converges to slightly worse final performance than Post-LN.

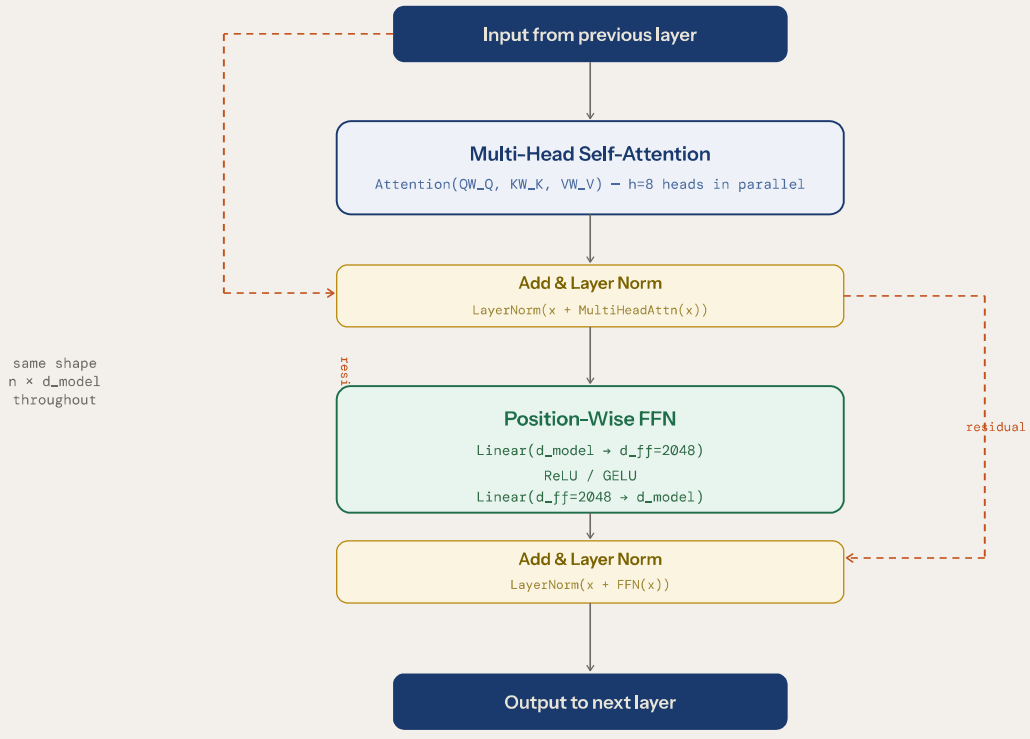

Figure 3 — A single encoder layer. Orange dashed lines are residual (skip) connections; the input bypasses each sub-layer and is added back to its output before normalisation.

06 STACKING ENCODER LAYERS

Depth and Representation

The full encoder consists of N identical layers stacked in sequence. Each layer refines the token representations produced by the layer below, building increasingly abstract and context-aware features.

In the original paper, `N = 6`. BERT-base uses `N = 12`, BERT-large uses `N = 24`. Larger models like Megatron-BERT use up to 72 encoder layers. Each layer receives the output of the previous layer as its input, and all layers share the same architecture — only the learned weights differ.

> WHAT DO LAYERS LEARN?
>
> Probing experiments and visualisations have revealed a consistent pattern across many encoder models: early layers (1–4) tend to capture low-level syntactic features — part-of-speech tags, basic morphology. Middle layers (5–8) represent higher-level syntax — dependency trees, constituent boundaries. Late layers (9+) encode semantic content — named entities, coreference, semantic roles. This hierarchical organisation mirrors the intuition behind deep learning generally: lower layers = generic features, higher layers = task-specific abstractions.

Parameter Count Analysis

For a model with `N` encoder layers, `d_model` dimensions, `h` heads, and FFN expansion factor 4:

COMPONENT	PARAMETERS PER LAYER	BERT-BASE (12 LAYERS)
QKV projections	`3 × d_model² / h × h = 3 × d_model²`	$3 \times 768^2 \approx 1.77M \times 12 = 21.2M$
Output projection W_O	`d_model²`	$768^2 \approx 0.59M \times 12 = 7.1M$
FFN W_1	`d_model × 4 × d_model`	$768 \times 3072 \approx 2.36M \times 12 = 28.3M$
FFN W_2	`4 × d_model × d_model`	$3072 \times 768 \approx 2.36M \times 12 = 28.3M$
LayerNorm (×2)	`2 × 2 × d_model`	negligible
Total (encoder only)	—	**≈ 85M**
+ Embeddings (30K vocab × 768)	—	+ 23M ≈ **108M total**

Dropout and Regularisation

The original Transformer applies dropout at three points in each encoder layer: to the attention weights (inside softmax), to the output of each sub-layer before the residual addition, and to the sums of the embeddings and positional encodings. Dropout rate is 0.1 in the base model. This prevents over-reliance on individual attention paths and improves generalisation.

Scaling Laws & Modern Variants

Understanding the computational and memory complexity of the encoder is essential for deploying and scaling Transformer-based models.

Complexity per Encoder Layer

OPERATION	TIME COMPLEXITY	MEMORY	PARALLELISABLE?
QKV Projections	$\mathcal{O}(n \cdot d_{\text{model}}^2)$	$\mathcal{O}(n \cdot d_{\text{model}})$	Yes
Attention Scores QK^T	$\mathcal{O}(n^2 \cdot d_k)$	$\mathcal{O}(n^2)$	Yes
Softmax + AV multiply	$\mathcal{O}(n^2 \cdot d_v)$	$\mathcal{O}(n^2)$	Yes
FFN (W_1, ReLU, W_2)	$\mathcal{O}(n \cdot d_{\text{model}} \cdot d_{\text{ff}})$	$\mathcal{O}(n \cdot d_{\text{ff}})$	Yes
Layer Norm (×2)	$\mathcal{O}(n \cdot d_{\text{model}})$	$\mathcal{O}(n \cdot d_{\text{model}})$	Yes

The dominant bottleneck for long sequences is the $\mathcal{O}(n^2)$ memory and compute of the attention matrix. For short sequences ($n < 512$), the FFN often dominates due to the large expansion factor. For very long sequences ($n \gg 1000$), the attention matrix overwhelms GPU memory.

Notable Encoder-Only Model Variants

MODEL	LAYERS	D_MODEL	HEADS	PARAMS	KEY INNOVATION
BERT-base	12	768	12	110M	MLM + NSP pre-training
BERT-large	24	1024	16	340M	Larger capacity
RoBERTa	24	1024	16	355M	Removes NSP; dynamic masking

MODEL	LAYERS	D_MODEL	HEADS	PARAMS	KEY INNOVATION
DeBERTa-v3	12/24	768/1024	12/16	184M+	Disentangled attn; ELECTRA pre-training
Longformer	12	768	12	148M	Sliding window + global attention for long docs
ELECTRA	12	256–768	4–12	14–335M	Replaced token detection; training efficiency

Attention-Efficient Variants

To overcome the `O(n²)` bottleneck, several approaches have been developed:

Sparse attention (Longformer, BigBird): tokens attend only to a local window and a small set of global tokens. $\mathcal{O}(n)$ per sequence instead of $\mathcal{O}(n^2)$.

Linear attention (Performer, Linformer): kernel-based approximations that replace the softmax with a linear operation, achieving $\mathcal{O}(n)$ complexity at the cost of some approximation error.

Flash Attention: not a different algorithm — still exact self-attention — but a hardware-aware implementation that tiles the computation to avoid materialising the full `n×n` matrix in HBM. Achieves 2–4× speedup with significantly lower memory.

---

08 IMPLEMENTATION REFERENCE

Building an Encoder in PyTorch

A clean, minimal implementation that makes the mathematical structure of the encoder concrete.

Scaled Dot-Product Attention

```python
import torch
import torch.nn as nn
import math

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Q: (batch, heads, seq_len, d_k)
    K: (batch, heads, seq_len, d_k)
    V: (batch, heads, seq_len, d_v)
    Returns: (batch, heads, seq_len, d_v)
    """
    d_k = Q.size(-1)

    # QK^T / sqrt(d_k) → shape: (batch, heads, seq, seq)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)

    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))

    attn_weights = torch.softmax(scores, dim=-1)

    return torch.matmul(attn_weights, V), attn_weights
```

Multi-Head Attention Layer

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_k = d_model // num_heads
        self.h   = num_heads

        # Combined QKV projection (3 × d_model² parameters)
        self.W_qkv = nn.Linear(d_model, 3 * d_model)
        self.W_o   = nn.Linear(d_model, d_model)
```

```
    def forward(self, x, mask=None):
        B, n, d = x.shape

        # Project and split into Q, K, V heads
        QKV = self.W_qkv(x).view(B, n, 3, self.h, self.d_k)
        Q, K, V = QKV.unbind(dim=2)
        Q = Q.transpose(1, 2)  # (B, h, n, d_k)
        K = K.transpose(1, 2)
        V = V.transpose(1, 2)

        # Attend in parallel across all heads
        attn_out, _ = scaled_dot_product_attention(Q, K, V, mask)

        # Concat heads and project back
        attn_out = attn_out.transpose(1, 2).contiguous().view(B, n, d)
        return self.W_o(attn_out)
```

Complete Encoder Layer

```
class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.attn   = MultiHeadAttention(d_model, num_heads)
        self.ffn    = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model),
        )
        self.norm1  = nn.LayerNorm(d_model)
        self.norm2  = nn.LayerNorm(d_model)
        self.drop   = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # Sub-layer 1: Multi-Head Attention + residual
        x = self.norm1(x + self.drop(self.attn(x, mask)))

        # Sub-layer 2: FFN + residual
        x = self.norm2(x + self.drop(self.ffn(x)))
        return x
```

```python
class TransformerEncoder(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads, d_ff,
                 num_layers, max_seq_len, dropout=0.1):
        super().__init__()
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_seq_len, d_model)  # learned PE
        self.layers  = nn.ModuleList([
            EncoderLayer(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])
        self.scale   = d_model ** 0.5
        self.drop    = nn.Dropout(dropout)

    def forward(self, token_ids, mask=None):
        B, n = token_ids.shape
        pos  = torch.arange(n, device=token_ids.device).unsqueeze(0)

        x = self.drop(self.tok_emb(token_ids) * self.scale
                      + self.pos_emb(pos))

        for layer in self.layers:
            x = layer(x, mask)

        return x  # (B, n, d_model)
```

USING BUILT-IN MODULES

In practice, PyTorch's `nn.TransformerEncoderLayer` and `nn.TransformerEncoder` implement the above with additional options (batch_first, norm_first for Pre-LN, fast-path via Flash Attention). For production use, HuggingFace's `transformers` library provides thoroughly tested, optimised encoder implementations for BERT, RoBERTa, DeBERTa, and others.

SUMMARY

The Encoder at a Glance

The Transformer encoder is a remarkably elegant architecture. Its power derives from three insights working in concert: the ability of self-attention to model arbitrary token relationships in parallel; the use of multiple independent attention heads to capture diverse types of linguistic structure; and the depth afforded by residual connections and layer normalisation, which enable stable training of networks with many layers.

From a sequence of discrete token IDs, the encoder produces a dense matrix of contextualised vector representations — one per token — that downstream tasks can consume directly. In encoder-only models like BERT, these representations have proven powerful enough to achieve state-of-the-art performance across virtually every natural language understanding benchmark when fine-tuned with task-specific heads.

The same core architecture, with varying hyperparameters, forms the backbone of models that process text, images (Vision Transformer), audio, protein sequences, and multimodal data. Understanding the encoder is understanding the foundation of modern deep learning for sequential data.